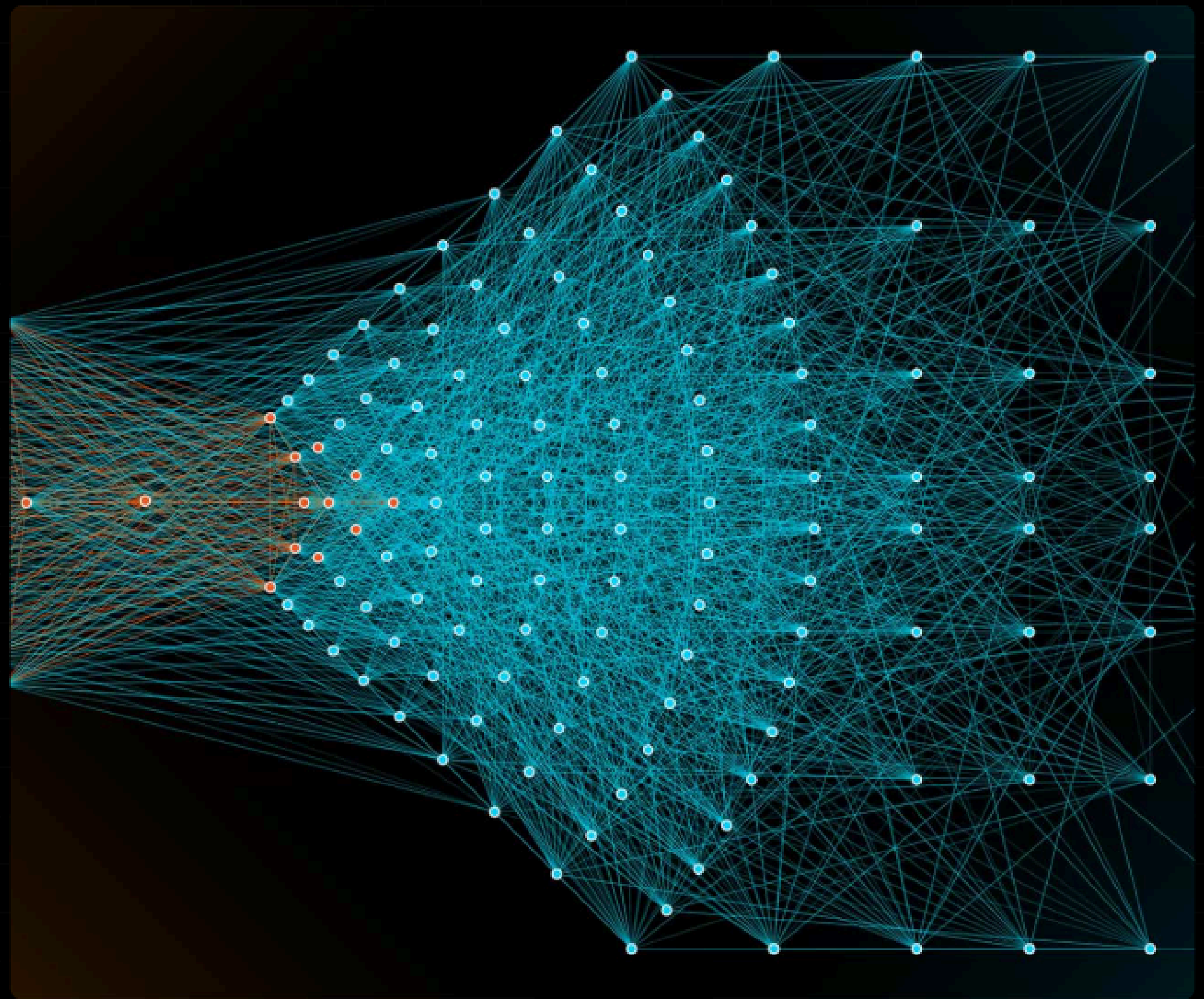


Building RAG with LangChain & Chroma

Implementing a complete RAG pipeline with LangChain,
Chroma, and Gradio for production environments
A comprehensive guide for engineers to master
retrieval-augmented generation.



What We're Building Today

A structured roadmap covering ingestion, storage, and deployment of RAG systems

1 RAG Fundamentals

2 LangChain Integration

3 Document Processing

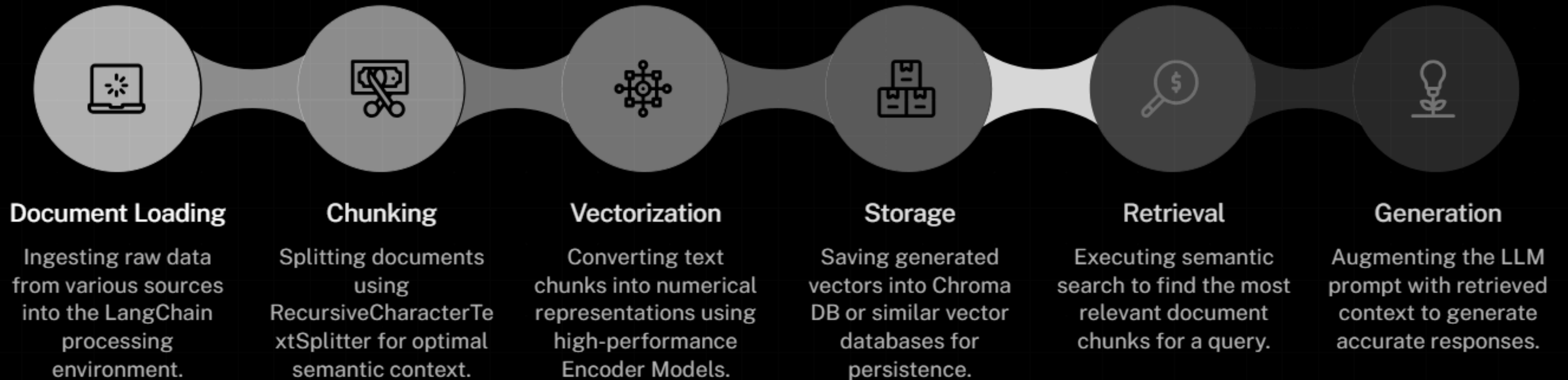
4 Embedding & Vectors

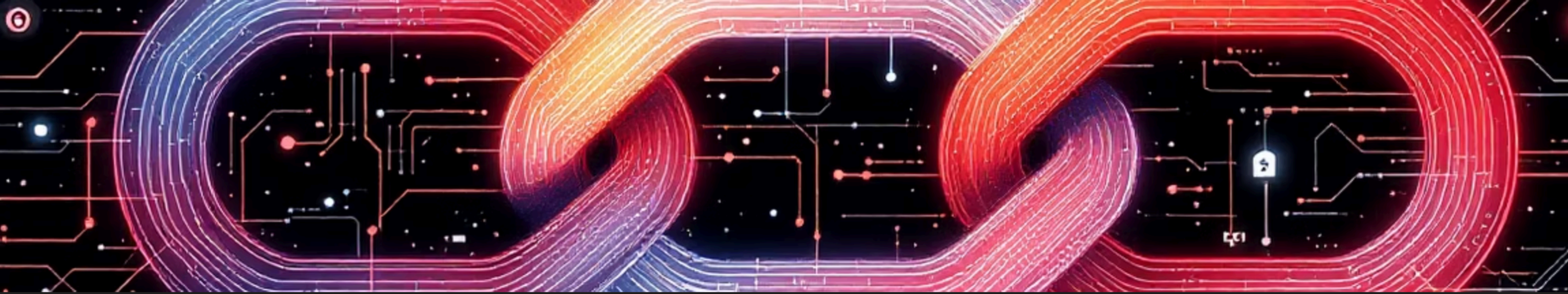
5 Pipeline Deployment

6 Best Practices

The RAG Implementation Flow

A step-by-step technical breakdown of how documents move through the retrieval-augmented generation pipeline.





Introducing LangChain

What is LangChain?

- Open source framework launched October 2022
- Created by Harrison Chase
- Abstraction layer for working with LLMs
- Helps build RAG pipelines, agents, and more
- Version 1.0 released October 2025 (major overhaul)

Website: <https://python.langchain.com/>



LangChain Advantages

Why LangChain remains the primary choice for engineering teams building complex AI applications.



Enables **rapid prototyping** and significantly faster time-to-market for AI features.



Provides **robust pre-built components** including loaders, splitters, and retrievers.



Benefits from **high enterprise adoption** and has become a verified industry standard.

LangChain Limitations

Understanding the trade-offs and architectural complexities when adopting the LangChain framework.



Features A **Steep Learning Curve** With Custom Terminology Like LCEL And HumanMessage.



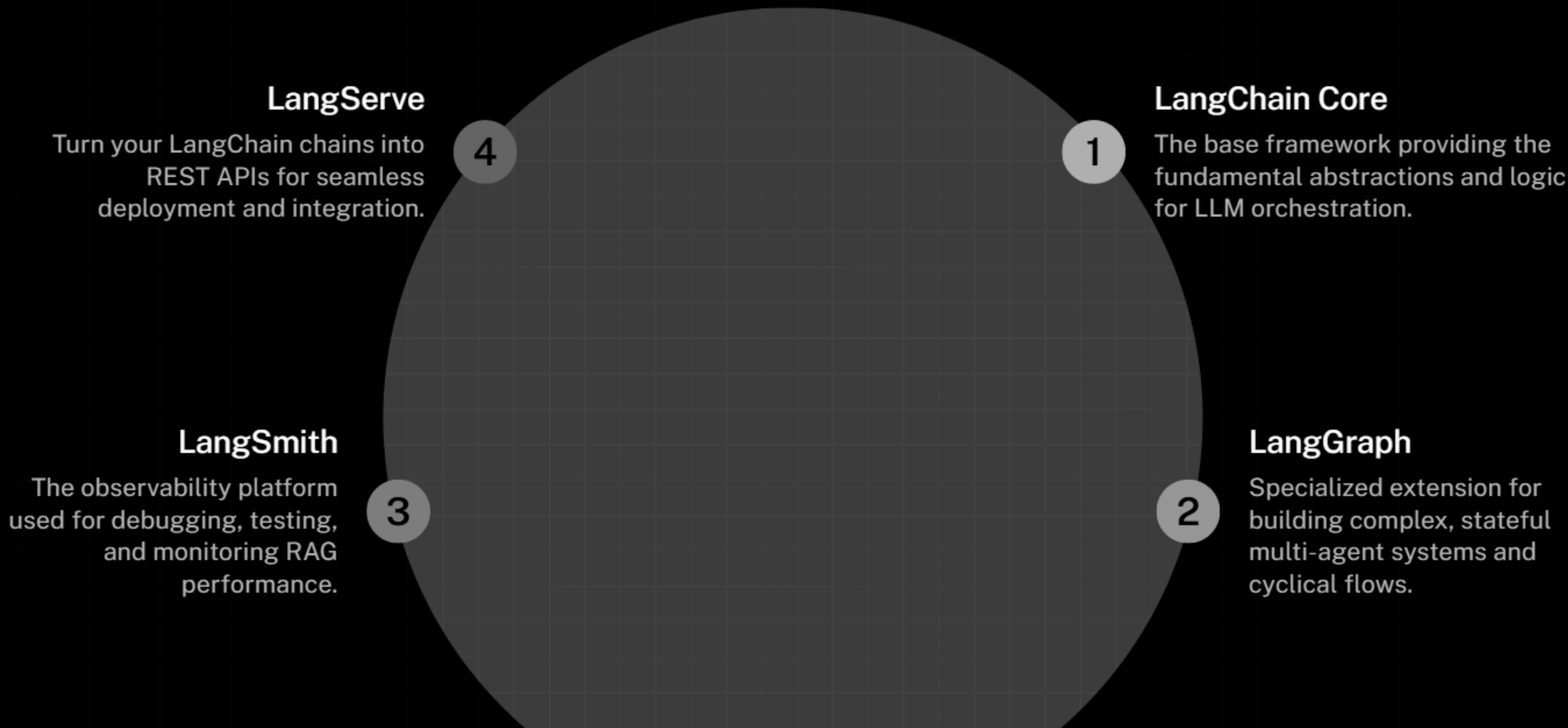
Maintains **Heavyweight Dependencies** That Can Complicate Lean Deployment Environments.



Direct API Calls May Be **More Efficient** For Simple Tasks Without Framework Overhead.

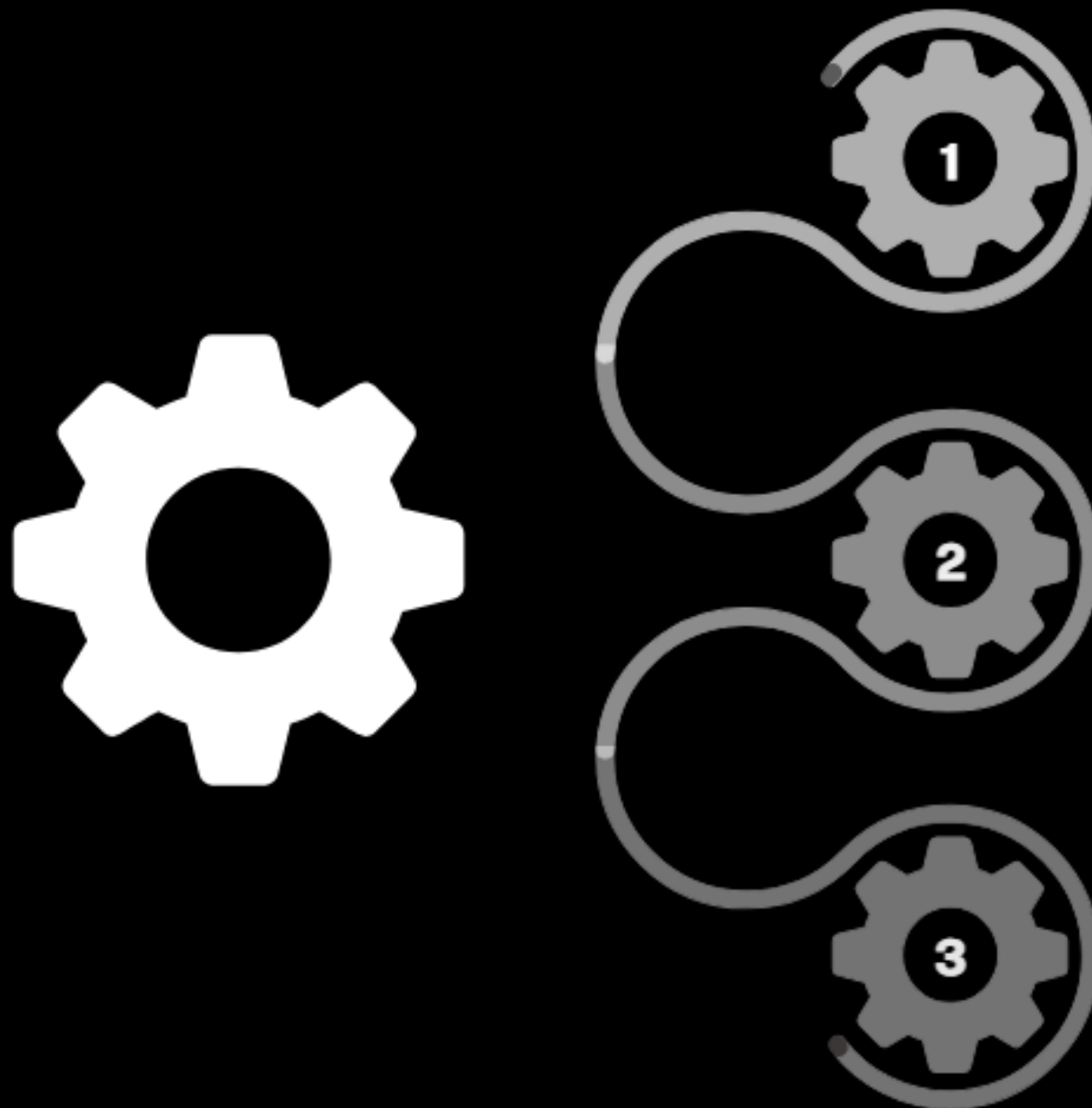
The LangChain Ecosystem

A modular suite of tools designed to handle every stage of the AI application lifecycle from development to deployment.



Why Document Chunking?

Strategic document splitting is essential to fit context windows and maintain semantic relevance in retrieval.



Context Limits

Large documents exceed LLM context windows; chunking creates manageable pieces of information.

Semantic Focus

Smaller chunks ensure that the retrieved context is highly specific to the user query.

Precision Retrieval

Enables the system to pinpoint the exact section of a document that answers the question.

Text Splitters in LangChain

Leveraging RecursiveCharacterTextSplitter to maintain document structure and semantic integrity during ingestion.



Splitting Hierarchy

- Primary split: **Double Newlines**
- Secondary split: **Single Newlines**
- Fallback: Sentences and Words



Recommended Parameters

- Chunk Size: **500-2000 chars**
- Overlap: **10-20%** of size

Why Chunk Overlap?

Chunk overlap is the primary defense against losing context during document splitting, ensuring that meaning is preserved at the boundaries where information would otherwise be severed.

An abstract graphic on a dark blue background featuring various 3D shapes and charts. In the top left, there's a bar chart with four bars of increasing height, colored red, orange, blue, and purple, labeled 2, 3, and 4. Below it is a line chart with blue bars. To the right is a large blue 3D pyramid. In the center, there's a bar chart with four bars labeled 01, 02, 03, and 04. Below that is a line chart with a blue line and an orange line. In the bottom left, there's a rainbow-like arch made of colored segments. In the bottom right, there's a pie chart with four segments of different colors. The overall theme is data visualization and analytics.

Encoder Models & Embeddings

Understanding the high-dimensional vector spaces that power semantic search in RAG pipelines.

384

Open Source Base

Standard dimensions for efficient models like all-MiniLM-L6-v2.

1,536

Proprietary Small

OpenAI text-embedding-3-small dimensions for better separation.

3,072

Proprietary Large

Max dimensions for OpenAI large models providing best semantic quality.

Popular Encoder Models

A comparative analysis of leading open-source and proprietary embedding models for production RAG.

Model Name	Provider	Dimensions	Key Performance
all-MiniLM-L6-v2	Hugging Face	384	Free, fast, decent separation
text-embedding-3-small	OpenAI	1536	Better separation, low cost
text-embedding-3-large	OpenAI	3072	Best quality, distinct clusters

Vector Databases

Choosing the right storage backend for indexing and querying high-dimensional vector embeddings.



Chroma DB is SQLite-based and serves as our primary easy-to-use recommendation.



FAISS offers high performance as an in-memory solution for fast retrieval.



Pinecone and **Weaviate** provide managed services for enterprise scaling.



Postgres and **MongoDB** now offer mainstream vector support for existing stacks.

Encoder vs Vector Database

Evaluating the relative impact of architectural choices on the final quality of retrieval.

”

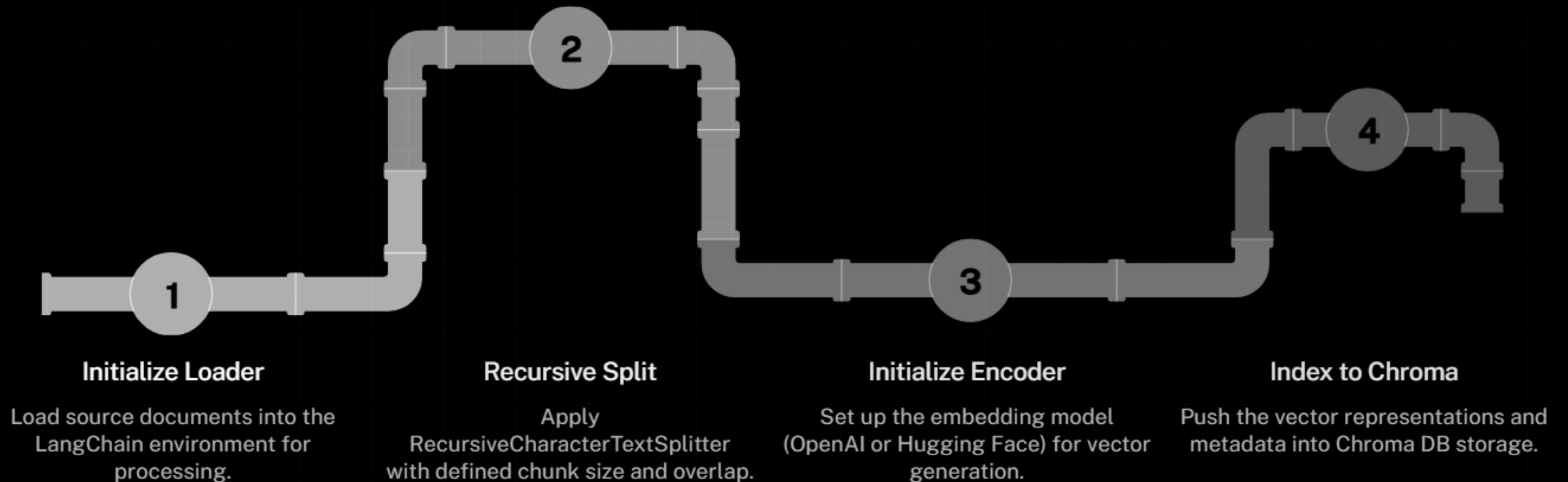
The **Encoder model choice** is more critical than the Vector Database choice for retrieval quality.

RAG Implementation Guide

Core Insight

Creating Vectors with LangChain

The technical sequence required to transform raw text into indexed vector data in Chroma DB.



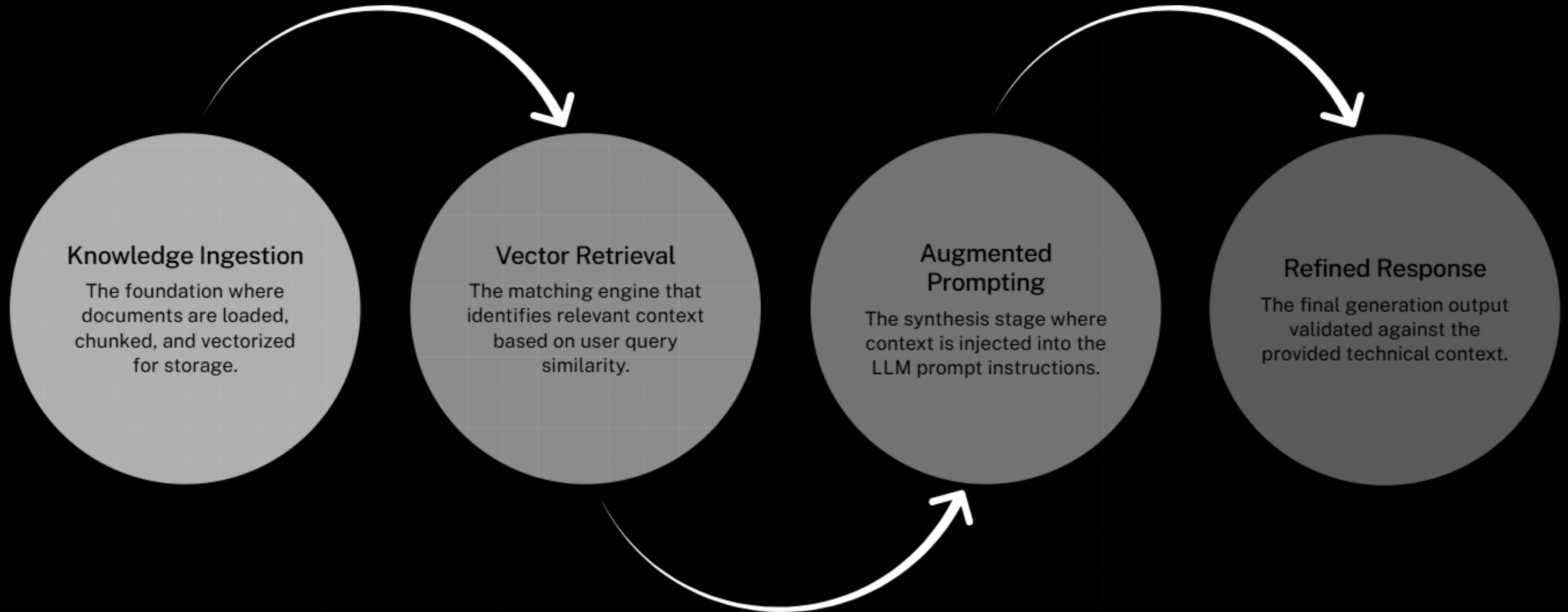


Visualizing Vectors

Using t-SNE dimensionality reduction to project high-dimensional data into human-interpretable spaces. Visualization via **t-SNE** allows us to project **384+ dimensions** down to 2D or 3D, making it possible to spot **semantic clusters** and data outliers in the vector space.

Building the RAG Pipeline

Integrating all components into a seamless, high-performance system for context-aware generation.



RAG in 5 Lines of Code

Demonstrating the power of LangChain abstractions: implementing a complete retrieval-augmented chain using just five fundamental lines of Python code.

```
# Fetch context
context = retriever.invoke(question)

# Build messages
messages = [
    SystemMessage(content=f"Context: {context}"),
    HumanMessage(content=question)
]

# Get answer
response = llm.invoke(messages)
```

Handling Conversation History

LLMs are inherently stateless; maintaining context requires manual management of the chat history. Because LLMs are **stateless**, conversation history must be **manually passed back** into the prompt for the model to handle context-aware follow-up questions effectively.

Production RAG Structure

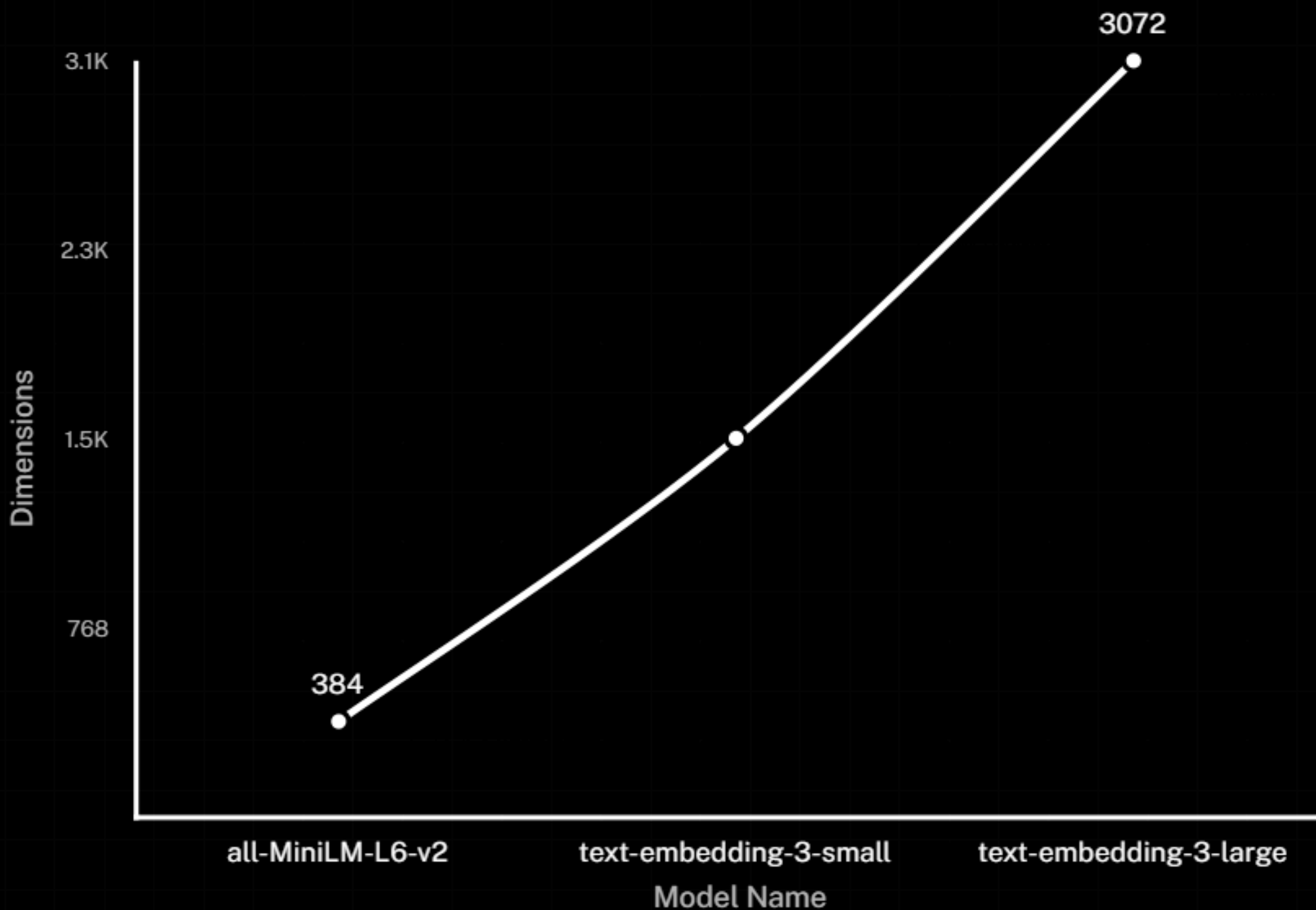
A modular file organization strategy to ensure code maintainability and swappable components.

File Name	Primary Responsibility	Core Actions
ingest.py	Data Processing	Loading, chunking , and vector storage
answer.py	Logic Core	Retrieval logic and LLM interaction
app.py	User Interface	Gradio UI and user interaction flow



Comparing Model Complexity

Visualizing the dimensionality differences between popular embedding models used in the course.



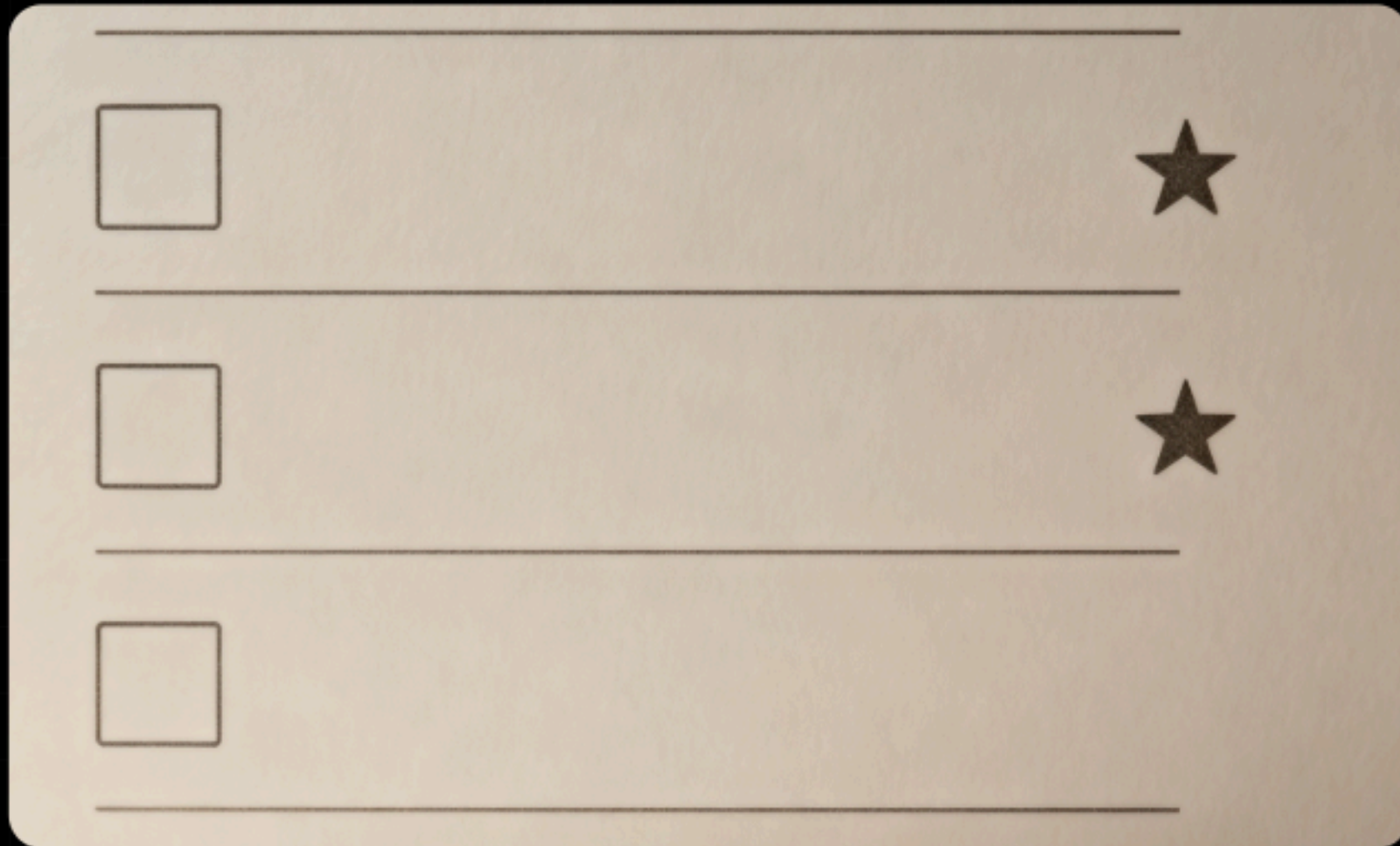
Key Insights from Visualization

Lessons learned from analyzing vector spaces and semantic clustering via t-SNE.

- Higher **dimensionality** results in more distinct semantic clusters.
- **t-SNE** reveals how the model groups similar concepts together visually.
- Outliers often indicate **poor quality chunks** or noise in the source data.

RAG Best Practices

Implementation recommendations derived from enterprise-level RAG development and testing.



Model Selection

- Start with **free open-source** (MiniLM) before scaling.
- Move to **OpenAI models** when higher accuracy is required.



Architectural Patterns

- Use **modular structures** (ingest/answer/app) for maintainability.
- Visualize vector spaces with **t-SNE** to debug issues.

1

Neglecting **chunk overlap**, leading to severed semantic context at boundaries.

2

Failing to pass **conversation history**, making follow-up questions impossible.

3

Using **improper chunk sizes** that are too small for context or too large for tokens.

Common Pitfalls

Avoiding frequent mistakes in document processing and retrieval logic that degrade system performance.



Key Takeaways

The most critical metrics and facts to remember from our Day 15 RAG implementation session.



Demo Total Chunks

413

Total chunks generated from the 76 demo documents provided.

Max Dimensions

3,072

Available in proprietary large models for maximum semantic quality.

Recommended Overlap

20 %

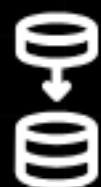
Target percentage of chunk size to prevent information loss.

Resources & Links

Official documentation and repositories to support your continued learning in the RAG ecosystem.



LangChain Documentation for core framework guides and API references.



Chroma DB Official Site for vector database installation and configuration.



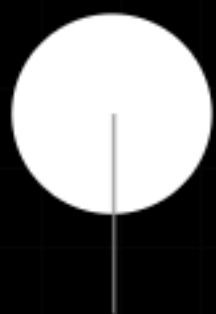
Hugging Face Model Hub to explore thousands of open-source encoders.



OpenAI Platform Guides for detailed specs on proprietary embedding models.

Next Steps

How to progress from this basic implementation to advanced RAG and production deployment.



Step 1

Refine Ingestion

Optimize chunking and overlap specifically for your unique dataset structure.



Step 2

Evaluation

Implement LangSmith to trace queries and evaluate retrieval accuracy.



Step 3

Advanced RAG

Explore multi-vector retrieval and parent-document retrieval strategies.



Step 4

Deployment

Containerize your app and deploy via LangServe as a robust production API.



The RAG Journey

Reflecting on the progress made from understanding concepts to building functional AI pipelines.



Foundation

Mastering LangChain and Chroma as the stable ground for your AI applications.

The Ascent

Navigating the complexities of chunking, vectorization, and model selection.

The Peak

Deploying a working RAG application that provides real value to users.

Digital You RAG

smqd19/**Digital-You-RAG**



 1
Contributor

 0
Issues

 0
Stars

 0
Forks



smqd19/Digital-You-RAG

Contribute to smqd19/Digital-You-RAG development by creating an account on GitHub.

 GitHub